Obelisk Proof Assistant — Reference Demonstration

Variant: Higher-Order Logic & Morse–Kelley Class Theory • Author: Georgy Dunaev • contact@obelisk.mozmail.com

1) Propositional logic

1.01) Axm. A01 (Weakening / Positive Implicative Axiom; K-Axiom; Russian: Утверждение следствия):
(α → (β → α))

1.02) Axm. A02 (Distribution of Implication):
((α → (β → γ)) → ((α → β) → (α → γ)))

1.03) Axm. A03 (Conjunction Elimination Left):
((α ∧ β) → α)

1.04) Axm. A04 (Conjunction Elimination Right):
((α ∧ β) → β)

1.05) Axm. A05 (Conjunction Introduction):
(α → (β → (α ∧ β)))

1.06) Axm. A06 (Disjunction Introduction Left):
(α → (α ∨ β))

1.07) Axm. A07 (Disjunction Introduction Right):
(β → (α ∨ β))

1.08) Axm. A08 (Disjunction Elimination / Cases):
((α → γ) → ((β → γ) → ((α ∨ β) → γ)))

1.09) Axm. A09 (Negation Introduction / Contradiction):
((¬α) → (α → ⊥))

1.10) Axm. A10 (Reductio Ad Absurdum):
((α → β) → ((α → ¬β) → ¬α))

1.11) Axm. A11 (Ex Falso Quodlibet):
(⊥ → α)

1.12) Axm. A12 (Law of Excluded Middle):
(α ∨ ¬α)

1.13) Rule Modus Ponens (MP):
α, (α → β) ⊢ β

2) Predicate logic

2.01) Axm. Discourse domain is non-empty:
∃(K : Class) ⊤

2.02) Rule Forall Instantiation (FI):
((∀(x : Class) φ) → φ[t/x])
Syntax side condition: FFI(t, v_x, φ)

2.03) Rule Exists Instantiation (EI):
(φ[t/x] → ∃(x : Class) φ)
Syntax side condition: FFI(t, v_x, φ)

2.04) Rule Forall Abstraction (FA / Bernays):
(ψ → φ) ⊢ (ψ → ∀(x : Class) φ)
Syntax side condition: (NFV(v_x, ψ) ∧ NFV(v_x, Γ))

2.05) Rule Exists Abstraction (EA / Bernays):
(φ → ψ) ⊢ (∃(x : Class) (φ → ψ))
Syntax side condition: (NFV(v_x, ψ) ∧ NFV(v_x, Γ))

2.06) Rule Generalisation (Gen):
φ ⊢ ∀(x : Class) φ
Syntax side condition: NFV(v_x, Γ)

3) Set/class-theoretic foundations of mathematics

3.01) Axm. Class Comprehension Axiom:

∀(x : Class) ((x ∈ { y | φ(y) }) ⟷ (isSet(x) ∧ φ(x)))

Syntax side condition: NFV(v_C, φ)

3.02) Axm. Extensionality Axiom:

∀(A, B : Class) ((∀(x : Class) ((x ∈ A) ⟷ (x ∈ B))) → (A = B))

3.08) Axm. Heredity Axiom (Subclasses of Sets are Sets):

∀(A, b : Class) ((isSet(b) ∧ (A ⊆ b)) → isSet(A))

3.12) Axm. Regularity (Foundation) Axiom:

∀(A : Class) ((A ≠ ∅) → ∃(x : Class) ((x ∈ A) ∧ ((x ∩ A) = ∅)))

3.13) Axm. Pairing Axiom:

∀(a, b : Class) ((isSet(a) ∧ isSet(b)) → isSet({a, b}))

3.14) Axm. Union Axiom:

∀(a : Class) (isSet(a) → isSet(⋃ a))

3.22) Def. IF-THEN-ELSE-FI (Class conditional with NFV side condition and context turnstile):

W, Γ ⊢ (⟦IF ψ THEN A ELSE B FI⟧ = { v | ((ψ ∧ (v ∈ A)) ∨ (¬ψ ∧ (v ∈ B))) })

Syntax side condition: (NFV(v_v, ψ) ∧ NFV(v_v, Γ))

3.23) Def. simple_if (metalevel conditional over Type_Expression):

∀(T_e : Type_Expression), (simple_if(⟦T_e⟧) : (Bool → (⟦T_e⟧ → (⟦T_e⟧ → ⟦T_e⟧))))

∀(T_e : Type_Expression), ∀(t₁ : ⟦T_e⟧) ∀(t₂ : ⟦T_e⟧) ((simple_if(⟦T_e⟧, true, t₁, t₂) = t₁) ∧ (simple_if(⟦T_e⟧, false, t₁, t₂) = t₂))

3.24) Def. if_dep (dependent conditional over Bool → Type_Expression):

∀(T_e : (Bool → Type_Expression)), (if_dep : Π(b : Bool), ⟦T_e(true)⟧ → (⟦T_e(false)⟧ → ⟦T_e(b)⟧))

∀(T_e : (Bool → Type_Expression)), ∀(b : Bool) ∀(t₁ : ⟦T_e(true)⟧) ∀(t₂ : ⟦T_e(false)⟧) ((if_dep(b, t₁, t₂) : ⟦T_e(b)⟧) ∧ ((if_dep(true, t₁, t₂) = t₁) ∧ (if_dep(false, t₁, t₂) = t₂)))

3.32) Thm. Ordinal Induction (∈-Ind):

((∀(x : Class) ((x ∈ On) → ((∀(y : Class) ((y ∈ x) → φ(y))) → φ(x)))) → ∀(x : Class) ((x ∈ On) → φ(x)))

Syntax side condition: (Fresh(v_y, v_x) ∧ NFV(v_y, φ(x)))

3.33) Def. Power Class Definition (□(A)):

∀(A, x : Class) ((x ∈ □(A)) ⟷ (isSet(x) ∧ (x ⊆ A)))

Syntax side condition: Fresh(v_x, v_A)

3.34) Axm. Power Set Axiom (Power Class of a Set is a Set):

∀(a : Class) (isSet(a) → isSet(□(a)))

3.37) Axm. Replacement / Limitation of Size Axiom:

∀(F, a : Class) ((isClassFunction(F) ∧ isSet(a)) → isSet(F[a]))

3.40) Def. Dunaev Tagged Ordered Pair:

∀(A, B : Class) ([A, B]_D = { z | ∃(a ∈ A) (z = {{{{a}}}, {∅}}) ∨ ∃(b ∈ B) (z = {{{{b}}}, {∅, {∅}}}) })

3.41) Thm. First Projection of Dunaev Tagged Ordered Pair:

∀(A, B : Class) ({ a | ({{{{a}}}, {∅}} ∈ [A, B]_D) } = A)

3.42) Thm. Second Projection of Dunaev Tagged Ordered Pair:

∀(A, B : Class) ({ b | ({{{{b}}}, {∅, {∅}}} ∈ [A, B]_D) } = B)

3.45) Axm. Infinity Axiom:

∃(x : Class) (isSet(x) ∧ ((∅ ∈ x) ∧ ∀(y : Class) ((y ∈ x) → ((y ∪ {y}) ∈ x))))

3.46) Axm. Global Choice Axiom:

∃(F : Class) (isClassFunction(F) ∧ ∀(x : Class) ((isSet(x) ∧ (x ≠ ∅)) → (F(x) ∈ x)))

3.47) Def. Universal Class (V):

(V = { x | (x = x) })

3.53) Def. Transitive Class Predicate:

(Tr(A) ⟷ ∀(x : Class) ((x ∈ A) → (x ⊆ A)))

3.66) Axm. Tarski's Axiom of Universes:

∀(x : Class) (isSet(x) → ∃(u : Class) ((isUniverse(u) ∧ (x ∈ u)) ∧ isSet(u)))

3.67) Def. Semi-Singleton Class Predicate (semi_singleton):

(semi_singleton(X) ⟷ ∀(u : Class) ∀(v : Class) (((u ∈ X) ∧ (v ∈ X)) → (u = v)))

3.68) Def. Value Extraction via Semi-Singleton Subclasses (sss / Value):
(sss(X) = ⋃({ z | ((z ⊆ X) ∧ semi_singleton(z)) }))

3.69) Thm. Theorem of Evaluation of a Family ((Λx, t)(z) = t[z/x]):
∀(z : Class) ((isSet(z) ∧ isSet(t[z/x])) → (((Λ(x : Class), t))(z) = t[z/x]))
Syntax side condition: FFI(z, v_x, t)

3.70) Def. Set-Theoretic Model of Dependent Sum (ΣB / Sigma Model):
(ΣB = { p | ∃(x : Class) ∃(y : Class) (((x ∈ Dom(B)) ∧ (y ∈ B(x))) ∧ (p = [x, y]_D)) })

3.71) Def. Set-Theoretic Model of Dependent Product (ΠB / Pi Model):
(ΠB = { f | ((isClassFunction(f) ∧ (Dom(f) = Dom(B))) ∧ ∀(x : Class) ((x ∈ Dom(B)) → (f(x) ∈ B(x)))) })

4) Metalogical laws

4.01) Axm. Instantiation Axiom:
∀(T_e : Type_Expression), ∀(G : Ctx) ∀(φ : Fm) ∀(t : ⟦T_e⟧) ∀(v : Variable) ((W, G ⊢ (∀(v : ⟦T_e⟧) φ)) → (W, G ⊢ (let v = t in φ)))

4.02) Axm. Generalisation Axiom:
∀(T_e : Type_Expression), ∀(Γ : Ctx) ∀(φ : Fm) ∀(v : Variable) (¬(Enc(v) ∈ Ctx.FV(Γ)) → ((W, Γ ⊢ φ) → (W, Γ ⊢ ∀(v : ⟦T_e⟧) φ)))
Syntax side condition: NFV(v, Γ)

4.03) Rule CoNecessitation:
(F, [] ⊢ ‹F, Γ ⊢ φ›) ⊢ (F, Γ ⊢ φ)

4.04) Def. Quantifier Unique (!):
∀(φ : Fm) ((!(z : Class) φ) ⟷ ∀(x, y : Class) ((φ[x/z] ∧ φ[y/z]) → (x = y)))

4.05) Def. Quantifier Exists and Unique (∃!):
∀(φ : Fm) (T, [] ⊢ ((∃!(x : Class) φ) ⟷ ((∃(x : Class) φ) ∧ !(x : Class) φ)))

4.06) Axm. Class Comprehension Axiom (Metatheory):
∀(φ : Fm) ∀(x : Name) ∀(k : Class.Expr) (FFI(k, x, φ) → (T, [] ⊢ ((k ∈ { x | φ }) ⟷ (isSet(k) ∧ φ[k/x]))))
Syntax side condition: FFI(k, x, φ)

4.07) Axm. Soundness / Reflection Axiom:
∀(φ : Fm) (⟦φ⟧ → (F, [] ⊢ φ))

4.08) Axm. Theory Inclusion (T ⊆ F):
∀(Γ : Ctx) ∀(φ : Fm) ((T, Γ ⊢ φ) → (F, Γ ⊢ φ))

4.09) Def. Syntactic Encoding Operator (Enc):
∀(T_e : Type_Expression), ∀(s : ⟦T_e⟧) (isSet(Enc(s)) ∧ (Enc(s) ∈ Emb(⟦T_e⟧)))

4.10) Def. Syntactic Decoding & Roundtrip Law (Dec):
∀(T_e : Type_Expression), ∀(s : ⟦T_e⟧) ((Dec(Enc(s), Emb(⟦T_e⟧)) : ⟦T_e⟧) = (s : ⟦T_e⟧))

4.11) Def. Dual Type Embedding Operator (Emb):
∀(X : Class) ((Emb(Class)(X) ⟷ ⊤) ∧ (Emb(Set)(X) ⟷ isSet(X)))

4.12) Def. Proof-Object Class Family (ProofsOf):
∀(s : Stmt) ∀(p : Prf) ((Enc(p) ∈ ProofsOf(Enc(s))) ⟷ CheckProof(p, s))

4.13) Def. Class of Provable Statements (Theorems):
∀(s : Stmt) ((Enc(s) ∈ Theorems) ⟷ (∃(p : Class) (p ∈ ProofsOf(Enc(s)))))

5) Tactics and inference rules

5.01) Thm. Elementary Theorems:
((F, [(b ∈ X)] ⊢ ∃(X : Class) (b ∈ X)) ∧ (F, [(b ∈ X)] ⊢ isSet(b)))

5.02) Rule Definition to Introduction Rule:
(F, [(b ∈ X)] ⊢ ∃(X : Class) (b ∈ X)) ⊢ (F, [(b ∈ X)] ⊢ isSet(b))

5.03) Rule Deduction Rule Application:
(F, [] ⊢ ((b ∈ X) → isSet(b)))

5.04) Thm. Tactics Proof Example:
F, [] ⊢ ((∃(Y : Class) ∀(x : Class) (x ∈ Y)) → ∀(x : Class) ∃(Y : Class) (x ∈ Y))

6) Incompleteness-related statements

6.01) Axm. Theories (Metalevel Stratification):

((Model(W) ⊆ Model(Q)) ∧ ((Model(Q) ⊆ Model(T)) ∧ (Model(T) ⊆ Model(F))))

6.02) Thm. Löb's Theorem:

∀(φ : Fm) (Q, [] ⊢ ((T, [] ⊢ ((T, [] ⊢ φ) → φ)) → (T, [] ⊢ φ)))

6.03) Thm. Gödel's Second Incompleteness Theorem:

Q, [] ⊢ (¬(T, [] ⊢ ⊥) → ¬(T, [] ⊢ ¬(T, [] ⊢ ⊥)))

Appendix: NBG Formulas-as-Types Standard Library

30 closed proof terms compiled by NBGSurfaceCompiler in Poly/ML. A formula is a type; a proof program is a term inhabiting it. The Dunaev pair injectivity theorem is the class-correct replacement for Kuratowski pairing on proper classes.

Theorem	Formula (Type)	AST	Norm
empty_class_no_elements	(∀(z : Class) ((z ∈ { x | ⊥ }) → ⊥))	7	7
empty_class_is_subclass	(∀(X : Class) (∀(z : Class) ((z ∈ { x | ⊥ }) → (z ∈ X))))	8	8
universal_class_characterization	(∀(z : Class) (((z ∈ { x | ⊤ }) → isSet(z)) ∧ (isSet(z) → (z ∈ { x | ⊤ }))))	15	15
russell_class_is_not_a_set	(isSet({ x | ((x ∈ x) → ⊥) }) → ⊥)	19	32
binary_inter_comm	(∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ { x | ((x ∈ A) ∧ (x ∈ B)) }) → (z ∈ { x | ((x ∈ B) ∧ (x ∈ A)) })))))	26	26
binary_union_comm	(∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ { x | ((x ∈ A) ∨ (x ∈ B)) }) → (z ∈ { x | ((x ∈ B) ∨ (x ∈ A)) })))))	23	23
class_diff_subclass_left	(∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ { x | ((x ∈ A) ∧ ((x ∈ B) → ⊥)) }) → (z ∈ A)))))	10	10
class_eq_sym	(∀(X : Class) (∀(Y : Class) ((X = Y) → (Y = X))))	6	6
class_eq_trans	(∀(X : Class) (∀(Y : Class) (∀(Z : Class) (((X = Y) ∧ (Y = Z)) → (X = Z)))))	9	9
quantifier_swap	((∃(y : Set) (∀(x : Set) (x ∈ y))) → (∀(x : Set) (∃(y : Set) (x ∈ y))))	7	7
dunaev_tagged_pair_projections	(∀(A : Class) (∀(B : Class) ((π₁([A, B]_D) = A) ∧ (π₂([A, B]_D) = B))))	3	3
cantor_diagonal_theorem	(∀(R : Class) (∀(c : Set) (isSet(c) → ((((c ∈ { x | (([x, x]_D ∈ R) → ⊥) }) → ([c, c]_D ∈ R)) ∧ (([c, c]_D ∈ R) → (c ∈ { x | (([x, x]_D ∈ R) → ⊥) }))) → ⊥))))	28	50
self_in_singleton	(∀(x : Class) (isSet(x) → (x ∈ { u | (u = x) })))	8	8
self_in_succ	(∀(x : Class) (isSet(x) → (x ∈ { u | ((u ∈ x) ∨ (u = x)) })))	9	9
class_sub_succ	(∀(x : Class) (∀(z : Class) ((z ∈ x) → (z ∈ { u | ((u ∈ x) ∨ (u = x)) }))))	12	12
dunaev_pair_injective	(∀(A : Class) (∀(B : Class) (∀(C : Class) (∀(D : Class) (([A, B]_D = [C, D]_D) → ((A = C) ∧ (B = D)))))))	26	26
dunaev_cartesian_elim	(∀(A : Class) (∀(B : Class) (∀(p : Class) ((p ∈ { q | (∃(u : Set) (∃(v : Set) (((u ∈ A) ∧ (v ∈ B)) ∧ (q = [u, v]_D)))) }) → (∃(u : Set) (∃(v : Set) (((u ∈ A) ∧ (v ∈ B)) ∧ (p = [u, v]_D))))))))	9	9
inter_sub_left	(∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ { u | ((u ∈ A) ∧ (u ∈ B)) }) → (z ∈ A)))))	10	10
inter_sub_right	(∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ { u | ((u ∈ A) ∧ (u ∈ B)) }) → (z ∈ B)))))	10	10
union_sub_left	(∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ A) → (z ∈ { u | ((u ∈ A) ∨ (u ∈ B)) })))))	13	13
semi_singleton_sub_elim	(∀(s : Class) (∀(A : Class) (∀(x : Class) (∀(y : Class) (((∀(u : Class) ((u ∈ s) → (u ∈ A))) ∧ (∀(u : Class) (∀(v : Class) (((u ∈ s) ∧ (v ∈ s)) → (u = v))))) → (((x ∈ s) ∧ (y ∈ s)) → ((x ∈ A) ∧ (x = y))))))))	19	19
family_fiber_char	(∀(A : Class) (∀(s : Class) ((s ∈ { u | (u c = A) }) → (isSet(s) ∧ (s c = A)))))	7	7
sigma_dependent_sum_intro	(∀(D : Class) (∀(Fiber : Class) (∀(x : Class) (∀(y : Class) ((isSet(x) ∧ ((x ∈ D) ∧ (y ∈ Fiber))) → (x ∈ { u | ((u ∈ D) ∧ (y ∈ Fiber)) })))))	9	9
pi_dependent_product_elim	(∀(D : Class) (∀(B : Class) (∀(x : Class) (((∀(u : Class) ((u ∈ D) → (u ∈ B))) ∧ (x ∈ D)) → (x ∈ B)))))	10	10
union_sub_right	(∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ B) → (z ∈ { u | ((u ∈ A) ∨ (u ∈ B)) })))))	13	13
model_stratification_trans	(∀(MW : Class) (∀(MQ : Class) (∀(MT : Class) (∀(MF : Class) (((∀(z : Class) ((z ∈ MW) → (z ∈ MQ))) ∧ ((∀(z : Class) ((z ∈ MQ) → (z ∈ MT))) ∧ (∀(z : Class) ((z ∈ MT) → (z ∈ MF))))) → (∀(z : Class) ((z ∈ MW) → (z ∈ MF))))))))	22	22
if_dep_true_elim	(∀(A : Class) (∀(T1 : Class) (∀(T2 : Class) (∀(x : Class) (((((x ∈ A) → (x ∈ T1)) ∧ (((x ∈ A) → ⊥) → (x ∈ T2))) ∧ (x ∈ A)) → (x ∈ T1)))))))	11	11
if_dep_false_elim	(∀(A : Class) (∀(T1 : Class) (∀(T2 : Class) (∀(x : Class) (((((x ∈ A) → (x ∈ T1)) ∧ (((x ∈ A) → ⊥) → (x ∈ T2))) ∧ ((x ∈ A) → ⊥)) → (x ∈ T2)))))))	11	11
inter_idempotent_elim	(∀(A : Class) (∀(z : Class) ((z ∈ { u | ((u ∈ A) ∧ (u ∈ A)) }) → (z ∈ A))))	9	9
singleton_membership_eq	(∀(a : Class) (∀(z : Class) ((z ∈ { u | (u = a) }) → (z = a))))	8	8

Reproduce locally: python3 obelisk_cli.py compile examples/nbg_verified_programs.nbg